

SERVIÇO PÚBLICO FEDERAL
UNIVERSIDADE FEDERAL DE SERGIPE
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA
COMPUTAÇÃO
TÓPICOS AVANÇADOS EM ENGENHARIA DE
SOFTWARE E SI I

Atividade 2

Implementação de Framework LLM-as-a-Judge e
Persistência em
Banco de Dados Relacional

Tutorial de Restore, Execução e Análise

Autor: Gilson Inácio da Silva
Disciplina: Tópicos Avançados em Engenharia de Software e SI I
Programa: Pós-Graduação em Ciência da Computação
Instituição: Universidade Federal de Sergipe - UFS
Equipe: Equipe 2 - Medicina
Domínio: Médico
Datasets: K-QA e USMLE com gabarito
Ano: 2026

São Cristóvão - SE

2026

Sumário

1	Introdução	3
2	Objetivo	3
3	Datasets utilizados	3
4	Artefatos herdados da Atividade 1	4
5	Arquitetura da solução	4
5.1	Visão geral do pipeline	4
5.2	Juiz neutro e justificativa técnica	5
6	Modelagem do banco de dados	5
6.1	Objetivo da modelagem	5
6.2	Tabelas principais	5
6.3	Relacionamentos essenciais	6
7	Estrutura do projeto	6
8	Rubrica do Juiz-IA	7
9	Prompt do Juiz	8
9.1	Idioma e estratégia	8
9.2	Estrutura conceitual do prompt	8
10	Execução do banco e carga inicial	9
10.1	Criação do banco	9
10.2	Execução do schema	9
10.3	Carga inicial de modelos e datasets	9
11	ETL da Atividade 1 para a Atividade 2	10
12	Execução do Juiz-IA com CLI robusta	10
12.1	Comportamento do script	10
12.2	Execução principal	10
12.3	Flags aceitas	11
13	Sanitização e limpeza do banco	11
14	Conversão da avaliação humana	11
15	Análise estatística e resultados	12
15.1	Execução da estatística	12
15.2	Resultados consolidados	12

15.3 Leitura metodológica dos resultados	12
15.4 Arquivos gerados na pasta outputs	13
16 Queries de análise	13
17 Backup e restore	13
17.1 Backup	13
17.2 Restore	14
18 Checklist final de auditoria	14
19 Conclusão	14

1 Introdução

Este tutorial documenta a versão **final e definitiva** da **Atividade 2** da disciplina **Tópicos Avançados em Engenharia de Software e SI I**, com foco na implementação de um framework *LLM-as-a-Judge* com persistência em **PostgreSQL** no **domínio médico**. O objetivo do projeto é transformar os artefatos produzidos na Atividade 1 em um experimento auditável, reproduzível, rastreável e estatisticamente analisável.

Na versão final do projeto, o backend heurístico e de teste foi abandonado. O ciclo definitivo passou a utilizar um **Juiz Neutro** de alta capacidade via API da Groq, operando com temperatura zero, cadeia de raciocínio explícita e rubrica clínica estruturada. Assim, o processo de auditoria deixou de ser apenas uma prova de conceito de pipeline para se tornar um fluxo completo de avaliação semântica com preservação de justificativa, nota, resultados estatísticos, backup e restore.

O presente material foi reescrito para refletir **apenas** a versão consolidada do projeto. Qualquer referência a modo **mock** deve ser entendida como parte de fases preliminares já superadas, não como parte da entrega final.

2 Objetivo

O objetivo central da Atividade 2 foi construir uma infraestrutura capaz de:

- armazenar datasets, perguntas, respostas candidatas, avaliações humanas e avaliações automatizadas em um banco relacional;
- implementar um juiz neutro para comparar respostas candidatas com uma referência padrão-ouro;
- registrar *Chain-of-Thought* do juiz e a nota final em escala de 1 a 5;
- calcular métricas de concordância entre avaliação humana e avaliação automática por meio da correlação de Spearman;
- garantir reprodutibilidade por meio de scripts, documentação, backup e restore do experimento.

Do ponto de vista de Engenharia de Software, a atividade consolida práticas de integração entre camada de dados, scripts de ETL, inferência com modelo externo, persistência de resultados e auditoria estatística.

3 Datasets utilizados

O experimento foi executado no domínio **médico**, reutilizando artefatos da Atividade 1. Dois conjuntos principais foram utilizados:

- **K-QA**: conjunto de questões abertas com resposta de referência;

- **USMLE com gabarito:** conjunto de questões de múltipla escolha com alternativa correta conhecida.

O recorte efetivamente consolidado na execução final foi composto por:

- **15 questões abertas** do K-QA, correspondentes aos IDs 86 a 100;
- **27 questões de múltipla escolha** do USMLE, correspondentes aos IDs 136 a 162;
- **45 respostas candidatas abertas** julgadas, oriundas da combinação de 15 questões abertas com 3 modelos candidatos.

A análise de correlação incidiu sobre as **45 respostas reais** do recorte aberto, pois é nesse subconjunto que existiam avaliação humana estruturada e resposta automatizada do juiz para comparação estatística.

4 Artefatos herdados da Atividade 1

A Atividade 2 não recriou as respostas da Atividade 1, mas sim herdou e reorganizou seus artefatos. Os principais insumos foram:

- curadoria das questões abertas em `curadoria_abertas.csv`;
- curadoria das questões de múltipla escolha em `curadoria_mc.csv`;
- respostas dos modelos candidatos em `respostas_llms.csv`;
- avaliações humanas estruturadas em `avaliacao_llms.csv`;
- arquivos-base `dataset.json`, `questions_w_answers.jsonl` e `usmle_questions.json`.

Esses arquivos foram normalizados e migrados para o PostgreSQL, tornando o experimento consultável por SQL e auditável por meio de relacionamentos explícitos.

5 Arquitetura da solução

5.1 Visão geral do pipeline

A solução final foi organizada em cinco blocos principais:

1. **Ingestão:** leitura dos artefatos da Atividade 1 por meio do ETL;
2. **Persistência relacional:** organização das entidades do experimento em PostgreSQL;
3. **Auditoria com Juiz Neutro:** chamada à API da Groq com o modelo `llama-3.3-70b-versatile`;
4. **Armazenamento das avaliações:** gravação da justificativa técnica e da nota atribuída pelo juiz;
5. **Análise estatística e evidências:** geração de CSVs, backup e restore.

5.2 Juiz neutro e justificativa técnica

A versão final abandonou o backend local de teste e adotou como juiz o modelo `llama-3.3-70b-versatile` consumido via API da Groq. Essa escolha foi motivada por três razões principais:

- **Capacidade:** trata-se de um modelo de classe alta, com 70 bilhões de parâmetros, mais adequado para avaliação crítica do que modelos compactos ou quantizados;
- **Neutralidade:** o uso de um modelo distinto dos candidatos reduz *Self-Preference Bias*, isto é, o risco de um modelo favorecer respostas semelhantes ao seu próprio estilo;
- **Determinismo:** a temperatura foi definida como zero, reduzindo variabilidade aleatória e aumentando a consistência da auditoria.

Essa estratégia é metodologicamente superior a usar GPT/Claude/Gemini para avaliarem a si mesmos, porque preserva a ideia de **Juiz Neutro** e evita contaminação por auto-preferência.

6 Modelagem do banco de dados

6.1 Objetivo da modelagem

A modelagem relacional foi planejada para registrar todo o ciclo de vida do experimento: origem da pergunta, dataset de procedência, resposta candidata, avaliação humana, modelo juiz, justificativa da avaliação e nota atribuída.

6.2 Tabelas principais

A Tabela 1 resume as entidades principais do banco e sua finalidade.

Tabela 1: Tabelas principais do experimento

Tabela	Finalidade
modelos	registra modelos candidatos e modelos-juiz, incluindo papel semântico no experimento.
datasets	armazena os datasets utilizados, distinguindo perguntas abertas e de múltipla escolha.
perguntas	guarda enunciado, resposta ouro e metadados da pergunta.
alternativas	persiste as opções das questões de múltipla escolha.
respostas_atividade_1	registra as respostas candidatas herdadas da Atividade 1.
avaliacoes_humanas_atividade_1	persiste a avaliação humana estruturada e sua conversão para a escala de 1 a 5.

Tabela	Finalidade
avaliacoes_juiz	registra a avaliação automatizada do juiz, incluindo nota, justificativa e data da auditoria.

6.3 Relacionamentos essenciais

Os relacionamentos principais do modelo são:

- cada registro em `perguntas` pertence a um dataset em `datasets`;
- cada resposta em `respostas_atividade_1` referencia uma pergunta e um modelo candidato;
- cada avaliação humana referencia exatamente uma resposta candidata;
- cada avaliação do juiz referencia uma resposta candidata e um modelo juiz.

Essa organização assegura rastreabilidade e permite reprocessar estatísticas sem ambiguidade.

7 Estrutura do projeto

A organização física da pasta `Atividade_2/` ficou conforme a estrutura consolidada do projeto final:

Listing 1: Estrutura do projeto final

```
Atividade_2/
|- README.md                # Este arquivo
|- .env                     # Chaves de API e configuracoes do banco
|- .env.example             # Exemplo de configuracao de ambiente
|- data/                    # Datasets e arquivos derivados da
    Atividade 1
|   |- avaliacao_llms.csv    # Avaliacoes humanas/curadas dos LLMs
|   |- curadoria_abertas.csv # Curadoria das questoes abertas K-QA
|   |- curadoria_mc.csv     # Curadoria das questoes de multipla
    escolha USMLE
|   |- dataset.json         # Dataset base de questoes abertas
|   |- questions_w_answers.jsonl # Questoes com respostas de referencia
|   |- respostas_llms.csv   # Respostas geradas pelos modelos
    candidatos
|   \- usmle_questions.json # Questoes USMLE
|- prompts/
|   \- juiz_medico_en.txt   # Rubrica medica robusta utilizada pelo
    juiz
|- sql/
|   |- 01_schema.sql        # DDL: tabelas, chaves e constraints
```

```

| |- 02_seed.sql                # DML: carga inicial de modelos e
|   datasets
| \- 03_queries_analise.sql    # Consultas analiticas
|- scripts/
| |- etl_atividade2.py         # Migracao dos artefatos da Atividade 1
| |- executar_juiz.py          # Motor de auditoria com CLI robusta
| \- calcular_correlacao.py    # Calculo das correlacoes estatisticas
|- outputs/
| |- avaliacoes_juiz.csv       # Exportacao bruta das notas do juiz
| |- pares_humano_vs_juiz.csv  # Pares comparativos entre avaliacao
|   humana e juiz
| |- correlacao_spearman.csv    # Metricas de correlacao de Spearman
| \- analise_erros.csv         # Analise qualitativa/quantitativa de
|   erros
|- backup/
| \- backup_atividade_2.sql     # Backup do banco de dados
\-- tutorial/
|   |- Tutorial_Atividade_2.tex
|   \- Tutorial_Atividade_2.pdf

```

A inclusão dessa estrutura no tutorial torna mais fácil a reprodução do experimento por terceiros e evita ambiguidades sobre o papel de cada pasta.

8 Rubrica do Juiz-IA

A rubrica do juiz utiliza escala de 1 a 5 e foi concebida para privilegiar **segurança clínica**, **adequação técnica** e **alinhamento com o padrão-ouro**. A qualidade literária da resposta não é tratada como o critério central; o fator dominante é o risco clínico associado ao conteúdo.

Tabela 2: Rubrica de avaliação do juiz

Score	Interpretação
1	erro crítico, conduta perigosa, dosagem errada, contraindicação ignorada ou omissão grave com potencial de dano.
2	resposta parcialmente correta, mas com falhas relevantes de segurança, exames obrigatórios omitidos, red flags ignoradas ou risco clínico importante.
3	resposta clinicamente aceitável, porém incompleta, genérica ou parcialmente desalinhada ao padrão-ouro.
4	resposta muito boa, tecnicamente adequada, segura e bem alinhada ao padrão-ouro.

Score	Interpretação
5	excelência clínica: resposta totalmente alinhada ou superior ao padrão-ouro em clareza, completude, segurança e precisão farmacológica.

9 Prompt do Juiz

9.1 Idioma e estratégia

No domínio médico, o prompt foi mantido em **inglês** para aproveitar melhor a padronização do raciocínio clínico, da terminologia biomédica e dos critérios de julgamento. Além disso, foi exigido que o modelo produzisse explicitamente:

- REASONING: justificativa técnica em inglês;
- SCORE: nota inteira de 1 a 5.

Essa abordagem implementa a técnica de *Chain-of-Thought*, tornando a avaliação do juiz auditável e passível de revisão humana posterior.

9.2 Estrutura conceitual do prompt

O prompt final contém os seguintes componentes lógicos:

- persona de avaliador médico sênior;
- objetivo de comparar resposta candidata com resposta ouro;
- contexto com pergunta, gold standard e resposta candidata;
- rubrica explícita de 1 a 5;
- formato obrigatório de saída com REASONING e SCORE.

Listing 2: Estrutura resumida do prompt medico em ingles

```
[PERSONA]
You are a senior medical evaluator and clinical educator...

[GOAL]
Evaluate a candidate AI response by comparing it against the gold-standard
answer.

[CONTEXT]
Medical Question / Clinical Case:
{question}

Gold-Standard Reference Answer:
{gold_answer}
```

Candidate AI Response:

{candidate_answer}

[SCORING RUBRIC]

Score 1: critical error / dangerous recommendation ...

Score 2: partially correct but unsafe or incomplete ...

Score 3: acceptable but incomplete ...

Score 4: very good and clinically sound ...

Score 5: excellent and fully aligned ...

[OUTPUT FORMAT]

REASONING: <technical justification in English>

SCORE: <integer from 1 to 5>

10 Execução do banco e carga inicial

10.1 Criação do banco

A primeira etapa operacional consiste em criar o banco PostgreSQL que receberá a modelagem relacional:

Listing 3: Criação do banco PostgreSQL

```
createdb -U postgres atividade2_med
```

10.2 Execução do schema

Depois disso, executa-se o esquema relacional com tabelas, chaves estrangeiras, restrições e índices:

Listing 4: Carga do schema relacional

```
psql -U postgres -d atividade2_med -f sql/01_schema.sql
```

10.3 Carga inicial de modelos e datasets

Em seguida, executa-se a carga inicial de modelos e datasets:

Listing 5: Carga inicial de modelos e datasets

```
psql -U postgres -d atividade2_med -f sql/02_seed.sql
```

Essa etapa registra os datasets utilizados e os modelos candidatos, além do modelo juiz usado pelo experimento final.

11 ETL da Atividade 1 para a Atividade 2

O script `etl_atividade2.py` foi responsável por migrar e normalizar os artefatos da Atividade 1 para a base relacional. Em termos funcionais, ele executa as seguintes tarefas:

- leitura dos arquivos CSV e JSON da pasta `data/`;
- identificação dos recortes válidos de K-QA e USMLE;
- inserção de perguntas e resposta ouro em `perguntas`;
- inserção das alternativas das questões de múltipla escolha em `alternativas`;
- inserção das respostas candidatas em `respostas_atividade_1`;
- inserção das avaliações humanas em `avaliacoes_humanas_atividade_1`.

O comando utilizado foi:

Listing 6: Execucao do ETL

```
python scripts/etl_atividade2.py
```

Após a carga, os valores consolidados do banco final ficaram assim:

- **42** perguntas;
- **134** alternativas;
- **45** respostas candidatas;
- **45** avaliações humanas.

12 Execução do Juiz-IA com CLI robusta

12.1 Comportamento do script

O script `executar_juiz.py` compõe o centro operacional da auditoria. Ele executa o seguinte fluxo:

1. conecta ao PostgreSQL;
2. localiza respostas ainda não julgadas pelo modelo juiz configurado;
3. monta o prompt do juiz com pergunta, resposta ouro e resposta candidata;
4. chama o backend configurado (na versão final, `custom` com Groq);
5. faz o parsing da saída para extrair `REASONING` e `SCORE`;
6. salva a avaliação na tabela `avaliacoes_juiz`.

12.2 Execução principal

Na versão final, a execução se dá por meio do comando:

Listing 7: Execucao do juiz com backend real

```
python scripts/executar_juiz.py --backend custom
```

12.3 Flags aceitas

O script também aceita opções úteis para auditoria incremental e inspeção sem escrita no banco:

Listing 8: Exemplos de flags da CLI

```
python scripts/executar_juiz.py --backend custom --limit 5
python scripts/executar_juiz.py --backend custom --dry-run
```

A flag `-limit` é útil para executar subconjuntos controlados da fila de avaliação. Já a flag `-dry-run` permite testar o fluxo sem persistir resultados.

13 Sanitização e limpeza do banco

Antes da geração dos resultados definitivos, foi necessário remover avaliações de teste herdadas de fases preliminares do projeto. Essa sanitização foi essencial para garantir que as métricas estatísticas finais refletissem apenas o **juiz real**.

A limpeza crítica consistiu em apagar registros de teste em `avaliacoes_juiz`, por meio de comando SQL no estilo:

Listing 9: Exemplo de limpeza dos registros de teste

```
DELETE FROM avaliacoes_juiz
WHERE id_modelo_juiz = (
    SELECT id_modelo
    FROM modelos
    WHERE nome_modelo = 'JUDGE_MODEL'
);
```

Com isso, os cálculos estatísticos finais passaram a considerar somente o juiz `llama-3.3-70b-versatile`.

14 Conversão da avaliação humana

Como a Atividade 1 utilizava uma escala total de 0 a 10 e a Atividade 2 padronizou o julgamento para a escala de 1 a 5, foi adotada a seguinte regra de conversão:

- 9–10 → 5
- 7–8 → 4
- 5–6 → 3

- $3-4 \rightarrow 2$
- $0-2 \rightarrow 1$

Essa conversão permite comparar diretamente notas humanas e notas automatizadas sem perder a ordenação ordinal necessária ao cálculo de Spearman.

15 Análise estatística e resultados

15.1 Execução da estatística

A análise estatística foi executada por meio do script:

Listing 10: Calculo das correlacoes

```
python scripts/calcular_correlacao.py
```

Esse script gera os pares humano versus juiz, calcula a correlação de Spearman em diferentes níveis de agregação e exporta os CSVs finais da pasta `outputs/`.

15.2 Resultados consolidados

A correlação de Spearman incidiu sobre **45 respostas reais** julgadas pelo juiz final. O resultado global consolidado foi:

- **n = 45**
- **Spearman Rho = 0.14**

Os resultados por modelo indicaram:

- Claude 4.6 Sonnet: correlação positiva fraca;
- Gemini 3.0: correlação positiva fraca, ligeiramente maior do que a de Claude;
- GPT-5.4 Thinking: variância zero nas notas, resultando em NaN para Spearman.

O caso do GPT merece interpretação específica: receber a mesma nota em todas as respostas significa alta homogeneidade do julgamento, o que pode ser entendido como consistência clínica segundo a rubrica utilizada. Em termos estatísticos, contudo, a ausência de variação inviabiliza o cálculo do coeficiente de Spearman.

15.3 Leitura metodológica dos resultados

O juiz `llama-3.3-70b-versatile` mostrou-se mais rigoroso do que a avaliação humana inicial, especialmente quando a resposta candidata apresentava omissões relevantes de segurança, condutas pouco específicas ou baixa aderência ao padrão-ouro. Isso é coerente com o fato de a rubrica priorizar risco clínico acima de fluidez textual.

Portanto, uma correlação global modesta não deve ser interpretada automaticamente como falha do sistema, mas como indício de que o juiz automatizado operou com um

padrão de segurança mais conservador.

15.4 Arquivos gerados na pasta outputs

A versão final da pasta `outputs/` deve conter quatro arquivos, resumidos na Tabela 3.

Tabela 3: Arquivos finais gerados em `outputs/`

Arquivo	Descrição
<code>avaliacoes_juiz.csv</code>	exportação bruta das notas, justificativas e metadados do juiz.
<code>pares_humano_vs_juiz.csv</code>	pares consolidados para comparação estatística entre avaliação humana e avaliação automatizada.
<code>correlacao_spearman.csv</code>	resultado agregado das correlações calculadas em diferentes níveis.
<code>analise_erros.csv</code>	casos relevantes de divergência entre avaliação humana e juiz, úteis para análise qualitativa.

16 Queries de análise

Além dos scripts Python, o projeto inclui um conjunto de consultas SQL em `03_queries_analise.sql`. Essas consultas foram pensadas para auditoria manual e inspeção direta do banco, permitindo obter respostas como:

- quantidade de perguntas por dataset;
- total de respostas por modelo candidato;
- média das notas do juiz por modelo;
- pares humano versus juiz;
- casos em que o juiz divergiu da avaliação humana.

Essas queries funcionam como camada de verificação independente em relação aos scripts Python, reforçando a auditabilidade do experimento.

17 Backup e restore

17.1 Backup

O backup final do experimento foi gerado com:

Listing 11: Geracao do backup final

```
pg_dump -U postgres -d atividade2_med -f backup/backup_atividade_2.sql
```

Esse arquivo garante a reconstituição completa do banco, incluindo schema, dados e avaliações persistidas.

17.2 Restore

O restore foi testado em um banco separado, chamado `atividade2_med_restore`, por meio dos comandos:

Listing 12: Restore do experimento em banco separado

```
createdb -U postgres atividade2_med_restore
psql -U postgres -d atividade2_med_restore -f backup/backup_atividade_2.sql
```

Após o restore, as contagens essenciais do banco restaurado coincidiram com a base original, demonstrando que o experimento pode ser reconstruído localmente de forma íntegra.

18 Checklist final de auditoria

Antes da entrega final, devem ser verificados os seguintes itens:

- o banco `atividade2_med` foi criado e populado corretamente;
- os scripts SQL de schema e seed executaram sem erro;
- o ETL migrou corretamente os dados da Atividade 1;
- o juiz real foi executado via backend `custom`;
- os registros de teste foram removidos antes da correlação final;
- os quatro arquivos finais da pasta `outputs/` estão presentes;
- o backup foi gerado com `pg_dump`;
- o restore foi validado em `atividade2_med_restore`;
- o README e o vídeo refletem a versão final com juiz real.

19 Conclusão

A Atividade 2 consolidou os artefatos da Atividade 1 em um framework auditável de avaliação semântica com persistência em banco relacional. A adoção de PostgreSQL garantiu rastreabilidade, integridade e capacidade de inspeção dos resultados; a adoção de um juiz neutro real via Groq tornou o pipeline metodologicamente mais robusto; e a geração de outputs, backup e restore assegurou reprodutibilidade.

A principal contribuição desta implementação final está em demonstrar que um experimento de *LLM-as-a-Judge* pode ser estruturado com rigor de Engenharia de Software,

contemplando modelagem relacional, scripts de ETL, avaliação automatizada auditável, análise estatística e restauração completa do ambiente experimental.